

MLOps Pipeline — Heart Disease Prediction

SANDIP BHATTACHARYYA (2025cs05025) · 6th May 2026

https://github.com/sandbh/ml_learning.git

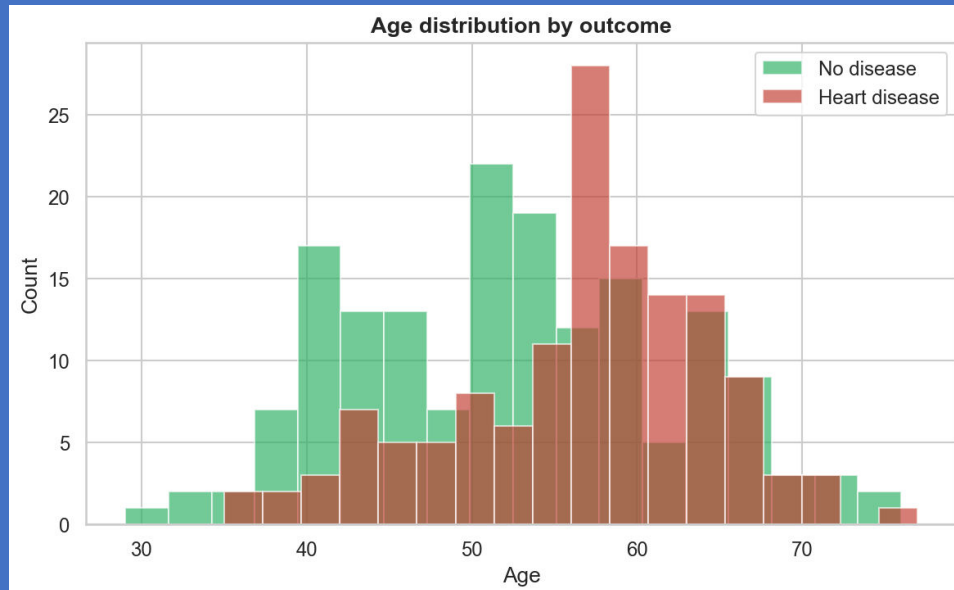
Table of Contents

1. Evidence & screenshots

A. EDA Plots

Age distribution by outcome

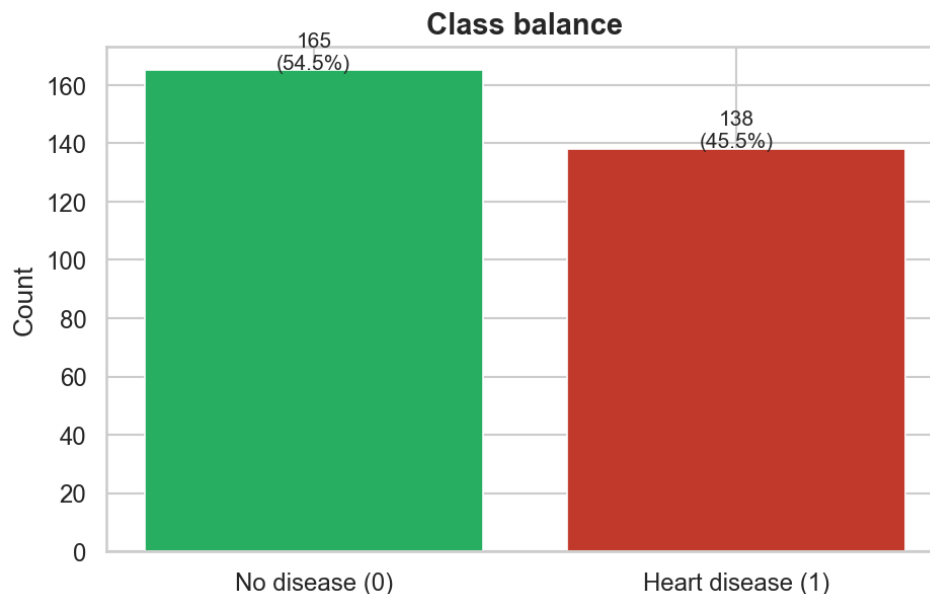
Overlapping histograms (or density plots) compare age distributions for patients with and without heart disease. Separation between curves hints at predictive signal; heavy overlap suggests age alone is not decisive—consistent with using the full feature vector in the model.



screenshots/age_by_target.png

Class balance of the target variable

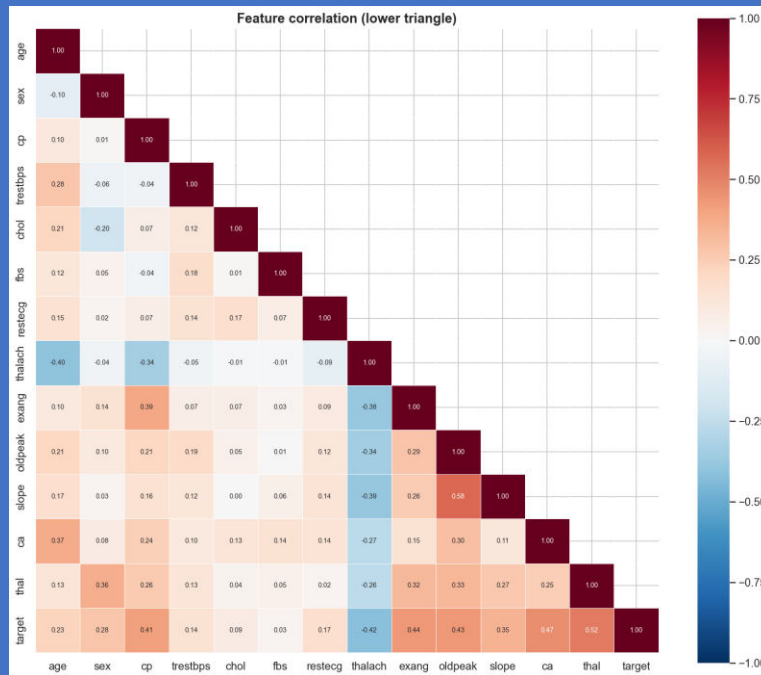
This chart summarises how many patient records belong to each outcome class (no disease vs heart disease). A severe imbalance would bias a classifier toward the majority class; here stratified train/test splitting and `class_weight` settings help mitigate that. Use this figure to justify preprocessing and evaluation choices.



screenshots/class_balance.png

Feature correlation heatmap

The lower-triangle Pearson heatmap highlights linear associations among features and with the target. Strong positive or negative blocks suggest redundant information or risk factors aligned with the label. This guides feature interpretation and complements model-based importance from Random Forest.



screenshots/correlation_heatmap.png

EDA / preprocessing console output

Terminal or log output from running the EDA CLI (for example python3 src/eda/eda.py all). It proves the pipeline executed successfully, shows paths to written CSVs and PNGs, and provides a timestamped audit trail for coursework.

```
[(.venv) (base) sandibha@SANDIBHA-M-QDX2 ml_learning % python3 src/eda/eda.py all]
=====
FULL PIPELINE - load, inspect, preprocess, EDA
=====
Dataset loaded successfully
Shape: (383, 14)
Columns: ('age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target')

First 5 rows:
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  ca  thal  target
0  63.0  male  angina  145.0  233.0  true  hyp  158.0  fal  2.3  slow  0.0  fix  H
1  67.0  male  asympt  148.0  286.0  fal  hyp  186.0  true  1.6  fast  3.0  norm  S2
2  67.0  male  asympt  128.0  229.0  fal  hyp  127.0  true  2.6  flat  2.0  rev  S1
3  37.0  male  notang  138.0  208.0  fal  norm  187.0  fal  3.5  down  0.0  norm  H
4  41.0  fem  abnang  138.0  284.0  fal  hyp  172.0  fal  1.4  up  0.0  norm  H

Data types:
age      object
sex      object
cp       object
trestbps object
chol     object
fbs      object
restecg  object
thalach  object
exang    object
oldpeak  object
slope    object
ca       object
thal     object
target   object
dtype: object

Basic statistics:
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  ca  thal  target
count  383  383  383  383  383  383  383  383  383  383  383  383  383
unique  41  2  4  49  152  2  3  91  2  48  3  5  4  5
top    55.0  male  asympt  128.0  284.0  fal  norm  152.0  fal  0.0  up  0.0  norm  H
freq    19  287  143  37  258  152  11  284  99  142  175  166  165

Missing / '?' tokens per column:
ca: 5
thal: 2

=====
Preprocessing (data_preprocessing/pre_processing_data.py)
=====
Filled missing in 'ca' with 0.0
Filled missing in 'thal' with 0.0

Cleaning done. Shape: (383, 14)
Target distribution:
target
0    165
1    138
Name: count, dtype: int64

=====
EDA
=====
Saved: /Users/sandibha/repos/ml_learning/screenshots/class_balance.png
Saved: /Users/sandibha/repos/ml_learning/screenshots/feature_histograms.png
Saved: /Users/sandibha/repos/ml_learning/screenshots/correlation_heatmap.png
Saved: /Users/sandibha/repos/ml_learning/screenshots/age_by_target.png

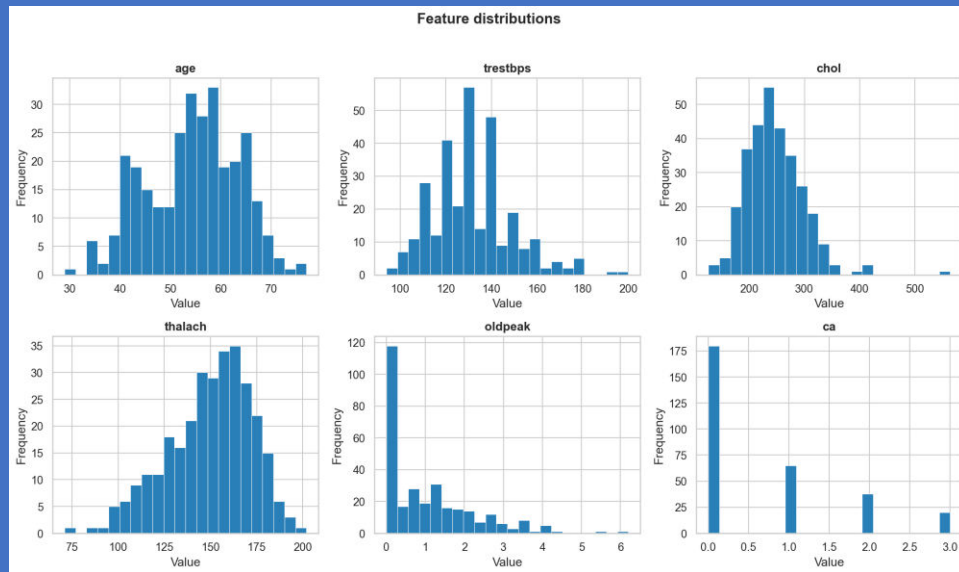
EDA plots saved under: /Users/sandibha/repos/ml_learning/screenshots

Cleaned dataset saved to /Users/sandibha/repos/ml_learning/data/heart_disease_processed_dataset.csv
[(.venv) (base) sandibha@SANDIBHA-M-QDX2 ml_learning %]
```

screenshots/eda_console_output.png

Feature histograms (numeric distributions)

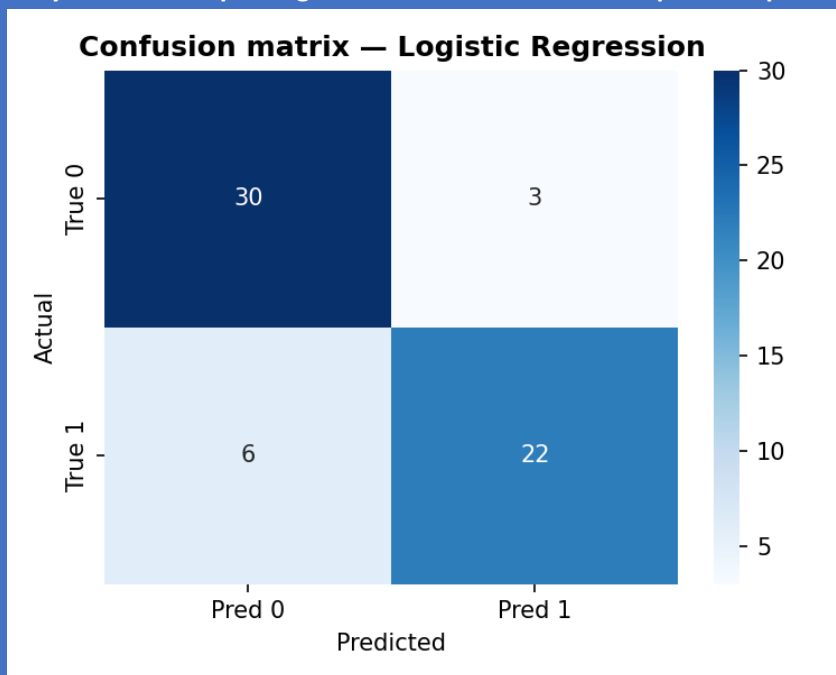
These histograms show how each numeric clinical input is distributed across patients. Skewed or multi-modal shapes motivate tree models or robust scaling; overlap between classes (not visible in a single panel) is explored in correlation and model evaluation plots. The figure supports the narrative that inputs are on different scales—hence StandardScaler inside the logistic pipeline.



screenshots/feature_histograms.png

Confusion matrix — Logistic Regression

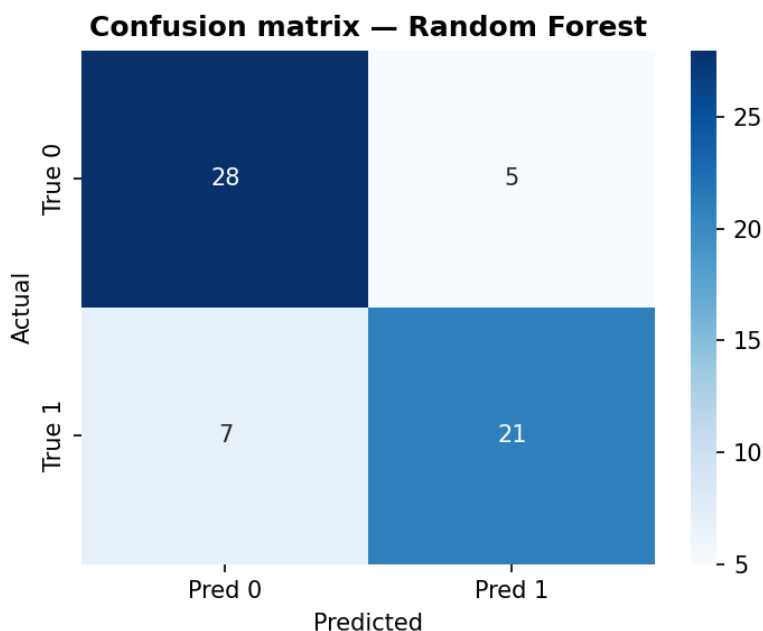
Rows represent the true label and columns the predicted label. Off-diagonal mass shows where the model confuses the two classes; diagonal dominance indicates good calibration for a binary screen. Compare against ROC for threshold-independent performance.



screenshots/cm_logistic_regression.png

Confusion matrix — Random Forest

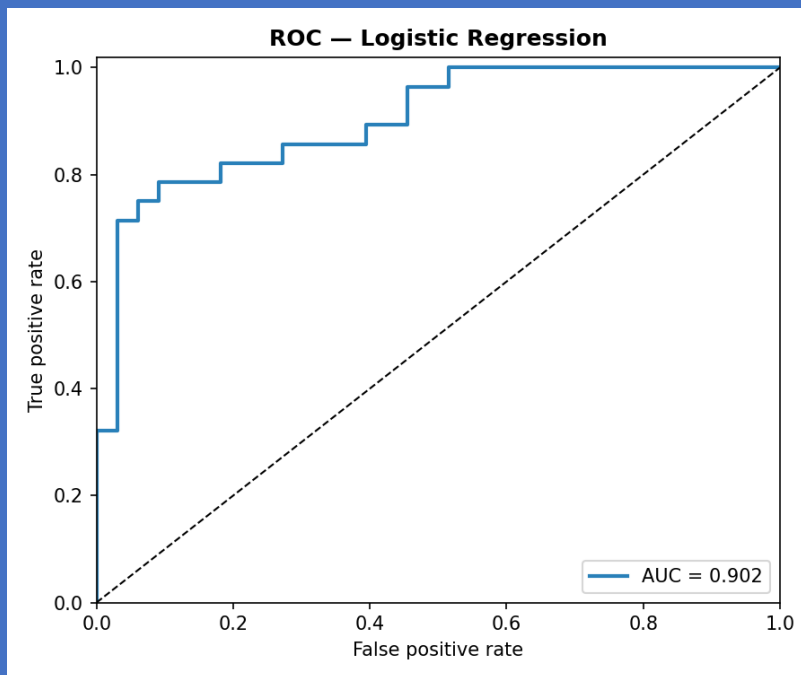
Rows represent the true label and columns the predicted label. Off-diagonal mass shows where the model confuses the two classes; diagonal dominance indicates good calibration for a binary screen. Compare against ROC for threshold-independent performance.



screenshots/cm_random_forest.png

ROC curve — Logistic Regression

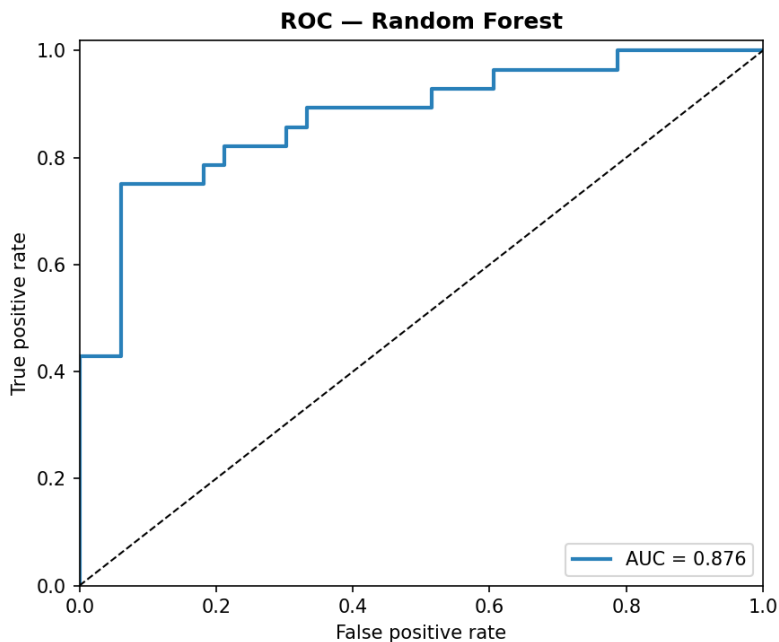
The receiver operating characteristic plots true positive rate vs false positive rate across thresholds. The area under the curve (ROC-AUC) is the primary metric used inside GridSearchCV; a curve bowed toward the top-left indicates strong discriminative ability on the hold-out set.



screenshots/roc_logistic_regression.png

ROC curve — Random Forest

The receiver operating characteristic plots true positive rate vs false positive rate across thresholds. The area under the curve (ROC-AUC) is the primary metric used inside GridSearchCV; a curve bowed toward the top-left indicates strong discriminative ability on the hold-out set.



screenshots/roc_random_forest.png

Console capture from `src/model_training/train.py` showing GridSearchCV progress, best hyperparameters, cross-validated metrics, hold-out test scores, and which model was persisted to `models/best_model.pkl`. This evidence ties the reported tables in section 4 to an actual run.

 Exchange Password Required
Enter your password for "sandibha" Internet Accounts.

screenshots/training_console_output.png

C. MLflow Tracking

MLflow experiment tracking UI

Screenshot of the MLflow UI listing runs for the heart-disease experiment. Each row corresponds to a model family or run with logged parameters and metrics; artifacts (curves, matrices) can be opened from the same interface. It demonstrates reproducible experiment management as required in MLOps coursework.

The screenshot shows the MLflow UI for the 'heart_disease_prediction' experiment. The table lists runs with the following columns: Run Name, Created, Dataset, Duration, Source, and Models. The runs are filtered by 'metrics.rmse < 1 and params.model = 'tree''. The table shows 18 runs, with a 'Show more columns (30 total)' button on the right.

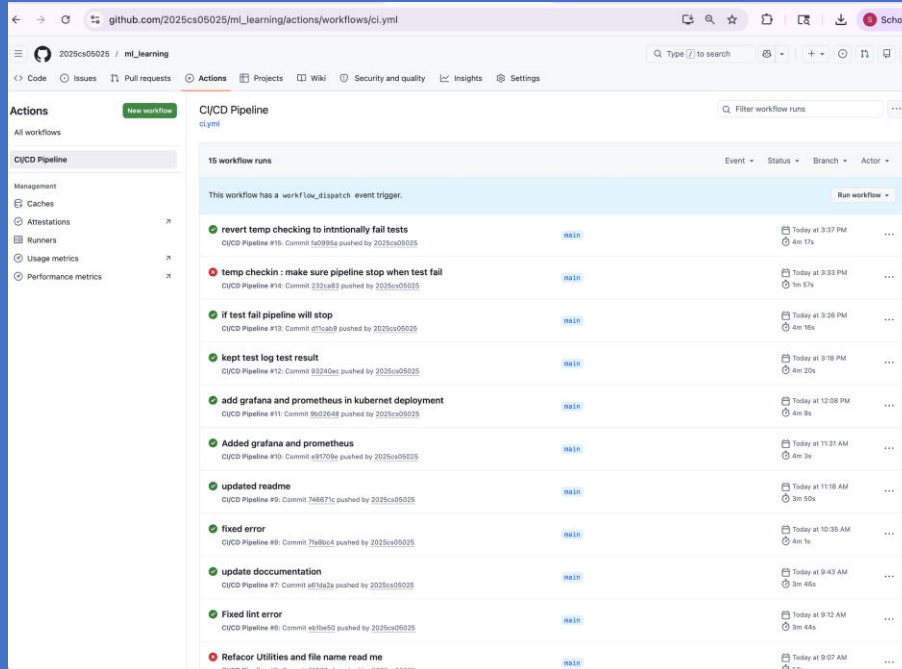
Run Name	Created	Dataset	Duration	Source	Models
Random Forest	5 minutes ago	-	1.3s	train.py	model
Logistic Regression	5 minutes ago	-	2.0s	train.py	model
Random Forest	9 hours ago	-	1.4s	train.py	model
Logistic Regression	9 hours ago	-	1.9s	train.py	model
Random Forest	9 hours ago	-	1.4s	train.py	model
Logistic Regression	9 hours ago	-	2.0s	train.py	model
Random Forest	9 hours ago	-	1.5s	train.py	model
Logistic Regression	9 hours ago	-	1.9s	train.py	model
Random Forest	9 hours ago	-	1.5s	train.py	model
Logistic Regression	9 hours ago	-	2.2s	train.py	model
Random Forest	10 hours ago	-	1.4s	train.py	model
Logistic Regression	10 hours ago	-	2.0s	train.py	model
Random Forest	10 hours ago	-	1.5s	train.py	model
Logistic Regression	10 hours ago	-	2.0s	train.py	model
Random Forest	10 hours ago	-	1.4s	train.py	model
Logistic Regression	10 hours ago	-	2.1s	train.py	model

screenshots/mlflows.png

D. CI/CD Pipeline

GitHub Actions CI/CD workflow run

This image shows a completed (or in-progress) workflow on GitHub: lint, test, training, and Docker smoke stages. Green checkmarks indicate automated quality gates; artifact uploads prove logs and model bundles were retained. It is primary evidence for the CI/CD section of the report.

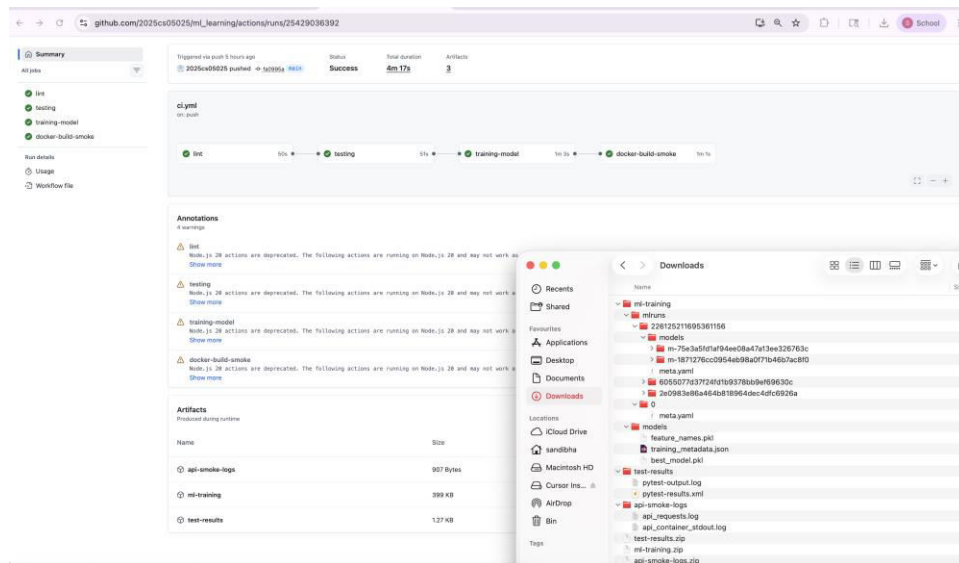


screenshots/CI_CD_pipeline.png

Artifacts downloaded from CI (metadata and logs)

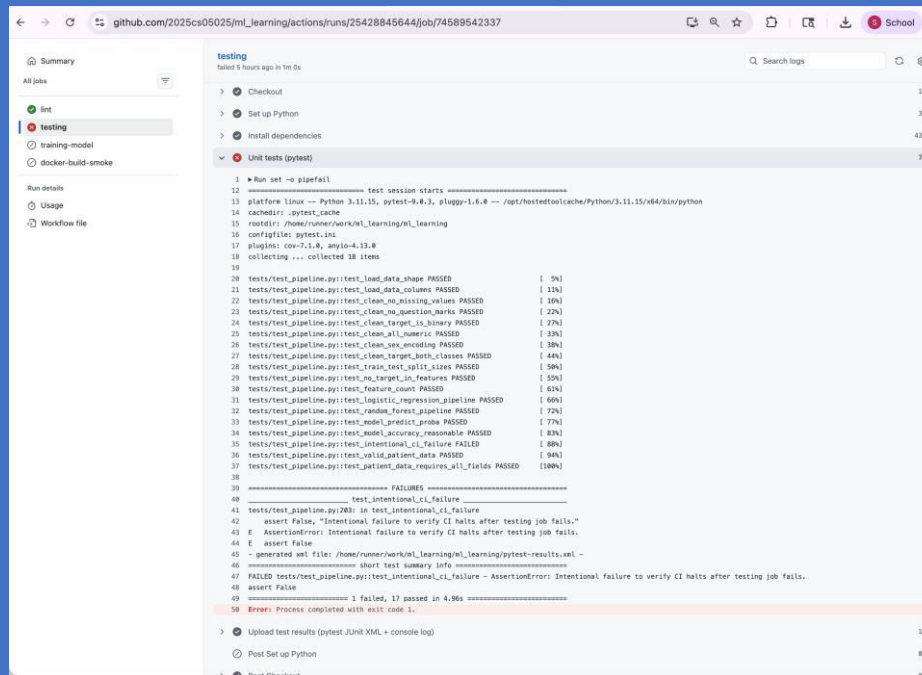
Evidence that training metadata, logs, or metrics were retrieved from the workflow artifacts.

This connects the cloud CI environment to the same files discussed in models/training_metadata.json and local reproducibility.



screenshots/metadata_and_logs_downloaded_from_Pipeline.png

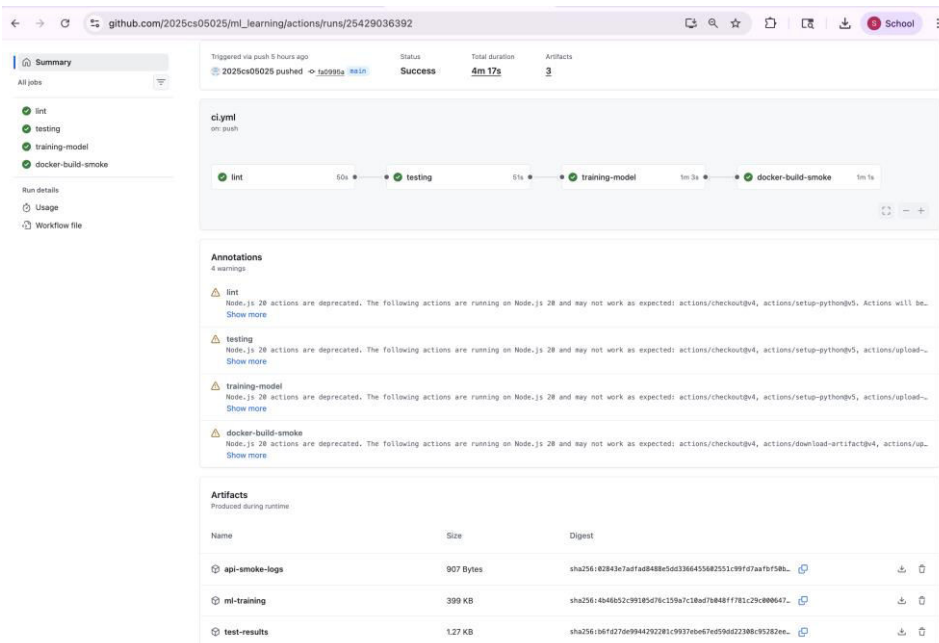
Pipeline behaviour after a failing test
Optional capture showing that later stages (train/Docker) did not run or were skipped after pytest failed. It illustrates fail-fast behaviour and is useful when discussing how CI protects main branch quality.



screenshots/pipeline_stop_after_test_failure.png

CI/CD stages with downloadable artifacts

Expanded view of pipeline jobs together with the Artifacts panel (test results, training bundle, API smoke logs). Downloadable artifacts are important for markers who want to verify pytest XML, training output, or container logs without re-running the pipeline locally.



screenshots/Successful_pipelinestages_with_artefacts.png

Docker image build log

Build output from docker build showing layer caching, dependency install, and successful image tagging. Together with a running container screenshot it proves the containerisation deliverable end-to-end.

```
(venv) (base) sandibha@SANDIBHA-M-QDX2 m1_learning % docker build -t heart-disease-api:latest .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 132.8MB
Step 1/11 : FROM python:3.11-slim
--> 6d85378d88a1
--> Using cache
--> e81388a4d78e
Step 2/11 : ENV PYTHONUNBUFFERED=1 PYTHONPATH=/app/src MODEL_PATH=/app/models/best_model.pkl FEATURE_PATH=/app/models/feature_names.pkl API_LOG_PATH=/app/logs/api_requests.log PIP_ROOT_USER_ACTION=ignore
--> Using cache
--> c59e71b7a4fa
Step 3/11 : RUN apt-get update && apt-get install -y --no-install-recommends gcc curl && rm -rf /var/lib/apt/lists/* && mkdir -p /app/logs
--> Using cache
--> c138e97e4d52
Step 4/11 : COPY requirements.txt .
--> Using cache
--> 480229e32802
Step 5/11 : RUN pip install --no-cache-dir --upgrade pip && pip install --no-cache-dir -r requirements.txt
--> Using cache
--> 6d4496aefcfa
Step 6/11 : COPY src/ ./src/
--> Using cache
--> faeae0932541
Step 7/11 : COPY models/ ./models/
--> Using cache
--> 8f6ae9fad8d3
Step 8/11 : EXPOSE 8080
--> Running in 3314080505a81
--> Removed intermediate container 3314080505a81
--> 82ae3ed969f9
Step 9/11 : HEALTHCHECK --interval=30s --timeout=10s --start-period=20s --retries=3 CMD curl -fs http://127.0.0.1:8080/health >/dev/null || exit 1
--> Running in 84562f39fedb
--> Removed intermediate container 84562f39fedb
--> bf1bde75362d
Step 10/11 : CMD ["uvicorn", "api.api:app", "--host", "0.0.0.0", "--port", "8080"]
--> Running in 41afd103092b
--> Removed intermediate container 41afd103092b
--> 7c277e9cdfce
Successfully built 7c277e9cdfce
Successfully tagged heart-disease-api:latest
(venv) (base) sandibha@SANDIBHA-M-QDX2 m1_learning %
```

screenshots/docker_build_log.png

Docker Compose build / pull output

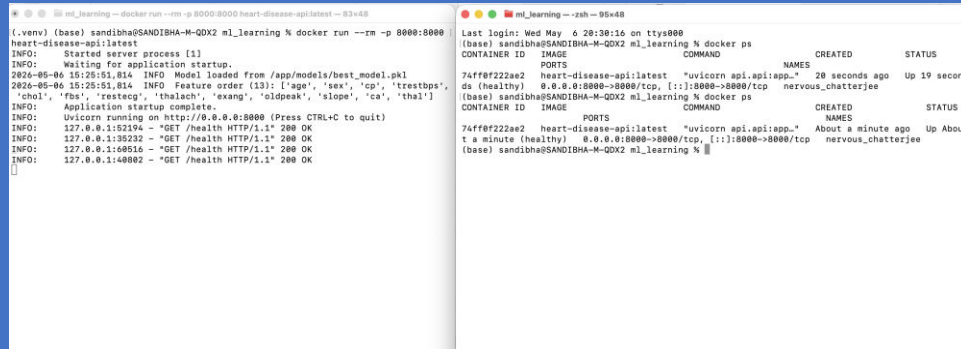
Console output while docker compose up --build brings up the API, Prometheus, and Grafana stack. It confirms service images built or pulled and that the multi-container definition in docker-compose.yml is valid.

```
(venv) (base) sandibha@SANDIBHA-M-QDX2 m1_learning % docker-compose up -d --build
[+] Building 0.0s
Sending build context to Docker daemon 37.2MB
Step 1/12 : FROM python:3.11-slim
--> 6d85378d88a1
--> Using cache
--> e81388a4d78e
Step 2/12 : ENV PYTHONUNBUFFERED=1 PYTHONPATH=/app/src MODEL_PATH=/app/models/best_model.pkl FEATURE_PATH=/app/models/feature_names.pkl API_LOG_PATH=/app/logs/api_requests.log PIP_ROOT_USER_ACTION=ignore
--> Using cache
--> c59e71b7a4fa
Step 3/12 : RUN apt-get update && apt-get install -y --no-install-recommends gcc curl && rm -rf /var/lib/apt/lists/* && mkdir -p /app/logs
--> Using cache
--> c138e97e4d52
Step 4/12 : COPY requirements.txt .
--> Using cache
--> 480229e32802
Step 5/12 : RUN pip install --no-cache-dir --upgrade pip && pip install --no-cache-dir -r requirements.txt
--> Using cache
--> 6d4496aefcfa
Step 6/12 : COPY src/ ./src/
--> Using cache
--> faeae0932541
Step 7/12 : COPY models/ ./models/
--> Using cache
--> 8f6ae9fad8d3
Step 8/12 : EXPOSE 8080
--> Using cache
--> 82ae3ed969f9
Step 9/12 : HEALTHCHECK --interval=30s --timeout=10s --start-period=20s --retries=3 CMD curl -fs http://127.0.0.1:8080/health >/dev/null || exit 1
--> Running in 84562f39fedb
--> Removed intermediate container 84562f39fedb
--> bf1bde75362d
Step 10/12 : CMD ["uvicorn", "api.api:app", "--host", "0.0.0.0", "--port", "8080"]
--> Running in 7c277e9cdfce
Step 11/12 : LABEL com.docker.compose.image.builder=classic
--> Running in 41afd103092b
--> Removed intermediate container 41afd103092b
--> 7c277e9cdfce
Successfully built 7c277e9cdfce
Successfully tagged m1_learning-heart-disease-api:latest
[+] up 0.0s
[+] Network m1_learning-heart-disease-api: Built
[+] Container heart-disease-api: Created
[+] Container heart-disease-api: Started
[+] Container prometheus: Created
[+] Container grafana: Created
(venv) (base) sandibha@SANDIBHA-M-QDX2 m1_learning % docker-compose ps
NAME                                SERVICE                                CREATED              STATUS              PORTS
heart-disease-api m1_learning-heart-disease-api uvicorn api.api:app 44 seconds ago      Up 44 seconds (healthy) 0.0.0.0:8080->8080/tcp
prometheus         grafana                                44 seconds ago      Up 44 seconds (healthy) 0.0.0.0:9090->9090/tcp
(venv) (base) sandibha@SANDIBHA-M-QDX2 m1_learning %
```

screenshots/Docker_compose_building.png

Running Docker container instance

Evidence of a live container (docker ps or Docker Desktop) exposing port 8000 for the API. This complements curl screenshots and shows the inference service is actually scheduled on the daemon.



The left terminal window shows the output of a Docker container running the 'heart-disease-api:latest' image. The logs indicate that the application started successfully, loaded the model, and is now running on http://0.0.0.0:8000. The right terminal window shows the output of the 'docker ps' command, which lists the container 'heart-disease-api:latest' with ID '7aff9f222ae2' and shows it is running and exposing port 8000.

```
(.venv) (base) sandibha@SANDIBHA-M-QDX2 m1_learning % docker run --rm -p 8000:8000 heart-disease-api:latest
INFO: Started server process [1]
INFO: Waiting for application startup.
2026-05-06 15:25:51.814 INFO Model loaded from /app/models/best_model.pkl
2026-05-06 15:25:51.814 INFO Feature order (13): ['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal']
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: 127.0.0.1:52194 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:35232 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:68516 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:48862 - "GET /health HTTP/1.1" 200 OK
```

```
Last login: Wed May 6 20:38:16 on tty4000
(base) sandibha@SANDIBHA-M-QDX2 m1_learning % docker ps
CONTAINER ID   IMAGE                                COMMAND                  NAMES                  CREATED          STATUS
7aff9f222ae2   heart-disease-api:latest            "uvicorn api.app:app-" 20 seconds ago       Up 19 seconds
ds (healthy)   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp  nervous_chatterjee
```

CONTAINER ID	IMAGE	COMMAND	NAMES	CREATED	STATUS
7aff9f222ae2	heart-disease-api:latest	"uvicorn api.app:app-"	nervous_chatterjee	20 seconds ago	Up 19 seconds (healthy)

(base) sandibha@SANDIBHA-M-QDX2 m1_learning % docker ps

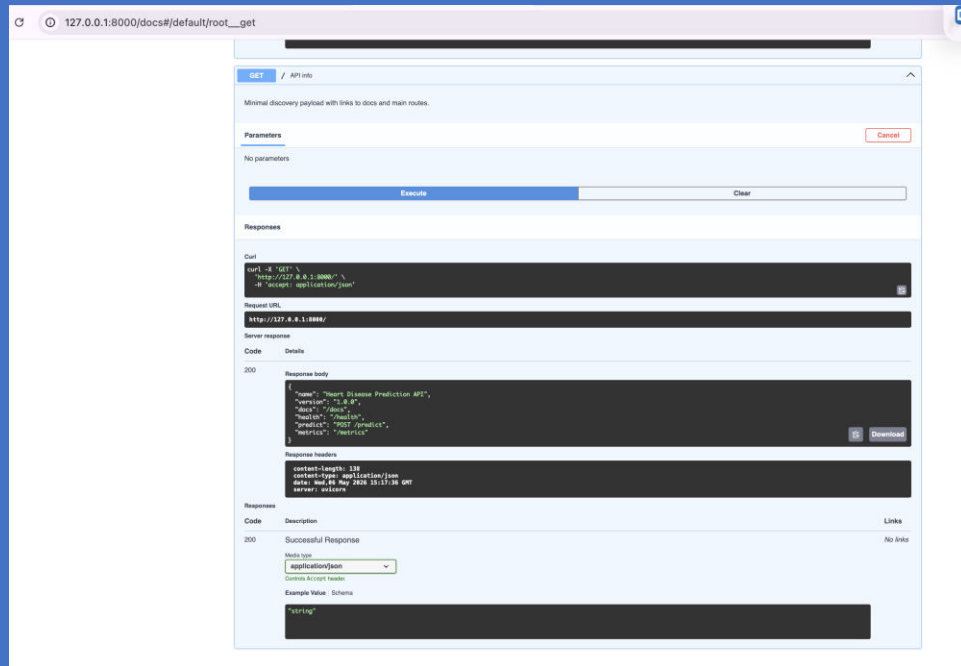
CONTAINER ID	IMAGE	COMMAND	NAMES	CREATED	STATUS
7aff9f222ae2	heart-disease-api:latest	"uvicorn api.app:app-"	nervous_chatterjee	About a minute ago	Up About a minute (healthy)

(base) sandibha@SANDIBHA-M-QDX2 m1_learning %

screenshots/Running_docker_instance.png

API health check (GET /health)

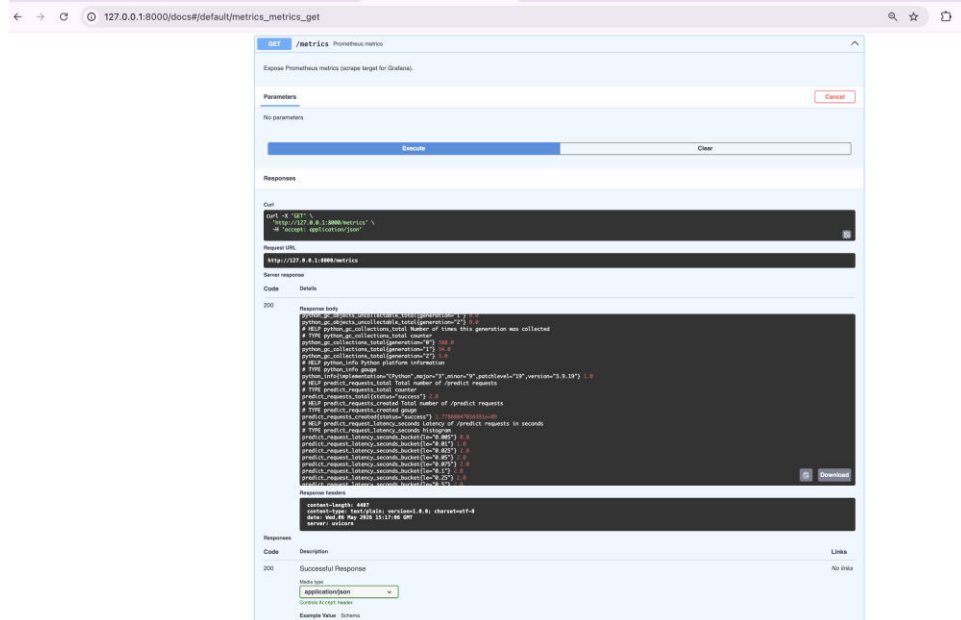
Browser or terminal output for GET /health returning JSON with healthy status and confirmation that the sklearn pipeline loaded. This is the liveness signal used by Docker HEALTHCHECK and Kubernetes probes.



screenshots/health_check_get.png

API / metrics or monitoring matrix view

Composite or dashboard-style capture relating health, metrics, or request paths—useful when demonstrating observability hooks alongside functional prediction tests.



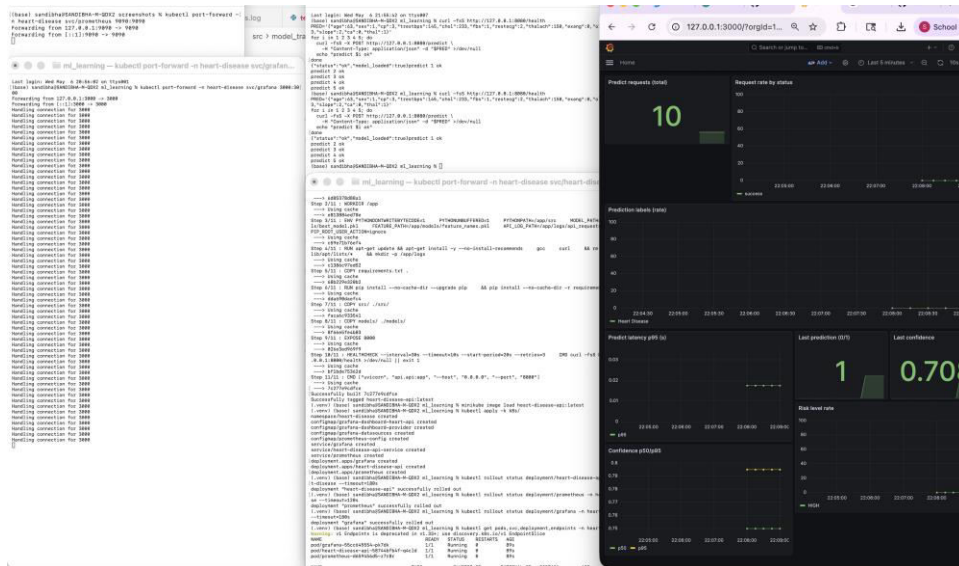
screenshots/health_check_matrix.png

Response payload from POST /predict showing prediction class, human-readable label, confidence, and risk band. It validates the contract expected by clients and matches the Pydantic schema tested in pytest.



Grafana with Kubernetes (Minikube) deployment

Dashboard or service view while the stack runs on Minikube, showing that monitoring was validated outside Docker Compose alone. Port-forwarding or ingress details may appear in the capture.



screenshots/grafana_prediction_data_with_minikube_deployment.png

Kubernetes (Minikube) deployment status

kubectl or dashboard output with pods/services in Running state for the API and optionally Prometheus/Grafana. Demonstrates the optional orchestration path (k8s/) required by advanced MLOps rubrics.

```

ml_learning -- ssh -- 260x98
[... (previous output) ...]
Step 4/11 : RUN apt-get update && apt-get install -y --no-install-recommends git curl && rm -rf /var/lib/apt/lists/* && mkdir -p /app/logs
--> Using cache
Step 5/11 : COPY requirements.txt .
--> Using cache
Step 6/11 : RUN pip install --no-cache-dir --upgrade pip && pip install --no-cache-dir -r requirements.txt
--> Using cache
Step 7/11 : RUN pip install --no-cache-dir --upgrade pip && pip install --no-cache-dir -r requirements.txt
--> Using cache
Step 8/11 : COPY models/ ./models/
--> Using cache
Step 9/11 : EXPOSE 8080
--> Using cache
Step 10/11 : CMD ["python3 main.py"]
--> Using cache
Step 11/11 : CMD ["python3 main.py"]
--> Using cache
Successfully built k8s/minikube
Successfully tagged heart-disease-api:latest
[... (previous output) ...]
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/heart-disease-api-service    ClusterIP           10.96.168.238    <none>            8080/TCP         89s
service/prometheus                   ClusterIP           10.96.139.46     <none>            9090/TCP         89s

NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/heart-disease-api    1/1      1              1            89s
deployment.apps/prometheus           1/1      1              1            89s

NAME                                ENDPOINTS           AGE
endpoints/heart-disease-api-service  10.244.0.22:8080    89s
endpoints/prometheus                 10.244.0.22:9090    89s
[... (previous output) ...]

```

screenshots/k8s_minikube_deployment.png

H. Evidence checklist (text logs & exports)

Evidence	Interpretation	Interpretation
age_by_target.png	Age distribution by outcome	Age distributions for no disease vs heart disease; overlap vs separation supports using the full feature set.
CICD_pipeline.png	GitHub Actions CI/CD workflow run	Workflow with lint, test, training, Docker smoke; checks and artifacts for the CI/CD section.
class_balance.png	Class balance of the target variable	Class counts; supports stratified splits and class_weight discussion.
cm_logistic_regression.png	Confusion matrix — Logistic Regression	True vs predicted; compare with ROC.
cm_random_forest.png	Confusion matrix — Random Forest	Same for random forest.
correlation_heatmap.png	Feature correlation heatmap	Pearson lower triangle; complements RF importance.
docker_build_log.png	Docker image build log	docker build output; use with a running-container shot.

File names	What it is	What it shows
Docker_compose_building.png	Docker Compose build / pull output	Compose bringing up API, Prometheus, Grafana.
eda_console_output.png	EDA / preprocessing console output	EDA CLI run log and paths to CSV/PNGs.
feature_histograms.png	Feature histograms (numeric distributions)	Numeric feature distributions; motivates scaling / model choice.
grafana_output_after_prediction.png	Grafana dashboard after predictions	Dashboard after traffic; ties to Prometheus /metrics .
grafana_prediction_data_with_minikube_deployment.png	Grafana with Kubernetes (Minikube) deployment	Grafana on Minikube; may show port-forward/ingress.
health_check_get.png	API health check (GET /health)	/health JSON; liveness for Docker/K8s.
health_check_matrix.png	API / metrics or monitoring matrix view	Health/metrics/paths in one view.
health_check_predict.png	API prediction request (POST /predict)	/predict response vs API contract / tests.
k8s_minikube_deployment.png	Kubernetes (Minikube) deployment status	Pods/services Running for API (and optional monitoring).
metadata_and_logs_downloaded_from_Pipeline.png	Artifacts downloaded from CI (metadata and logs)	CI-downloaded metadata/logs vs training_metadata.json .
mlflows.png	MLflow experiment tracking UI	MLflow runs, params, metrics, artifacts.
pipeline_stop_after_test_failure.png	Pipeline behaviour after a failing test	Fail-fast when tests fail.
pushing_data_from_api_before_grafana_output.png	Traffic generation before Grafana visualisation	Load so Grafana/Prometheus show non-zero series.
roc_logistic_regression.png	ROC curve — Logistic Regression	ROC-AUC and threshold behaviour.
roc_random_forest.png	ROC curve — Random Forest	Same for random forest.

Filename	Role	Description
Running_docker_instance.png	Running Docker container instance	Live container / port 8000.
Successful_pipeline_stages_with_artefacts.png	CI/CD stages with downloadable artifacts	Jobs + Artifacts (tests, bundle, logs).
training_console_output.png	Model training console	train.py : GridSearchCV, metrics, best_m

Filename	Description
api_container_stdout.log	Container/API stdout log
api_requests.log	API request log
docker_logs.txt	Docker-related text log;
heart_disease_processed_dataset.csv	Processed dataset export from EDA;
k8s_minikubelog.txt	Kubernetes/Minikube text log
pytest-output.log	Pytest text output
pytest-results.xml	JUnit-style test results (e.g. for CI).